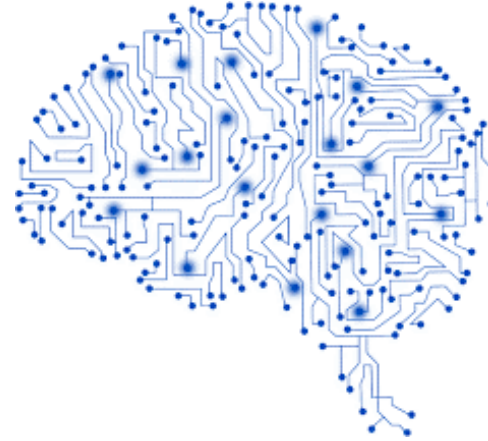




University of Minho
School of Engineering



Dados e Aprendizagem Automática

Reinforcement Learning:

Q-Learning and SARSA

DAA 2025/2026

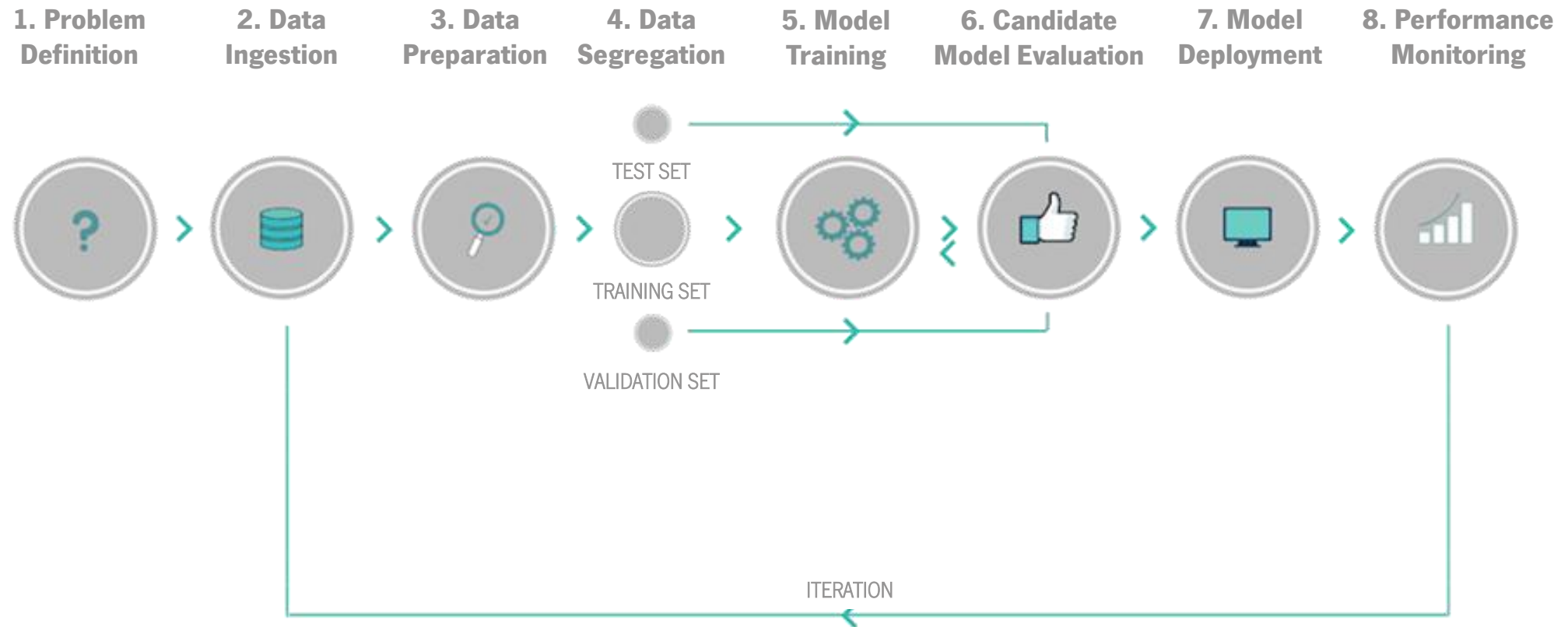
Filipa Ferraz, Victor Alves, Dalila Durães, Francisco Marcondes, Guilherme Barbosa

Part XII

Contents

2

- Reinforcement Learning
 - Q-Learning
 - SARSA
- Hands On

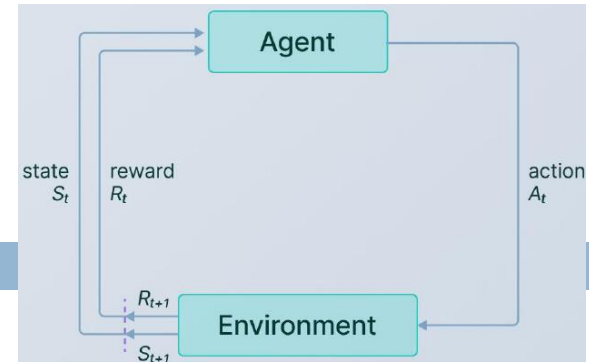




Reinforcement Learning

Reinforcement Learning

5



- **Agent (A)**: an entity that performs actions in an environment to gain a reward;
 - **Environment (e)**: a scenario that an agent encounters;
 - **Reward (R)**: an immediate return given to the agent for performing a specific action or task;
 - **State (S)**: the current situation returned by the environment;
 - **Policy (π)**: the strategy that the agent uses to decide on the next action based on the current state;
 - **Value (V)**: the expected long-term return discounted compared to the short-term reward;
 - **State-value Function $V(s)$** : indicates, for each state s , the expected return if the agent starts in that state and follows a given policy (e.g., expected chance of victory in a chess game given the current state);
 - **Action-value Function $Q(s, a)$** : (*Q-value*) indicates the expected value of taking an action a in a state s and then following the given policy (e.g., it can quantify the different possible moves in a chess game given the current state).
-
- **Model-Free RL**: (e.g., video games) helps you to make inferences and determine how the environment will behave. It learns policies or value functions directly from observed transitions and rewards (Q-Learning, for example);
 - **Model-based RL**: (e.g., drone navigation) it first learns a model of the environment's dynamics and then uses this model to plan and simulate future actions.

Reinforcement Learning

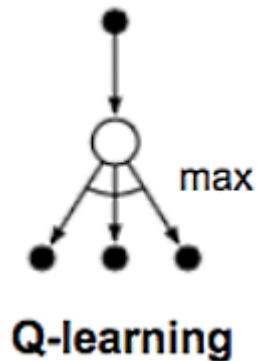
6

Problem: develop an intelligent bot capable of making decisions to solve a specific problem.

Model: **Reinforcement Learning (RL)** algorithms:

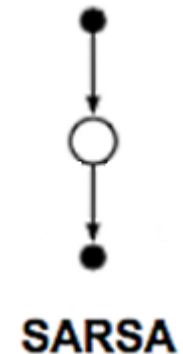
- **Q-Learning**, which is an off-policy and greedy learner.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \underbrace{\gamma \max_a Q(s_{t+1}, a)}_{\text{max}} - Q(s_t, a_t)]$$



- **SARSA**, a on-policy learner.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \underbrace{\gamma Q(s_{t+1}, a_{t+1})}_{\text{SARSA}} - Q(s_t, a_t)]$$



Reinforcement Learning

7

In order to implement the models, some packages need to be installed:

- [Gymnasium](#): an open-source Python library for developing and comparing RL algorithms by providing a standard API for communication between learning algorithms and environments, as well as a standard set of environments that comply with that API.
- [Pyglet](#): a Python library for developing games and other visually rich applications.
- [Pygame](#): a set of Python modules designed for writing games.

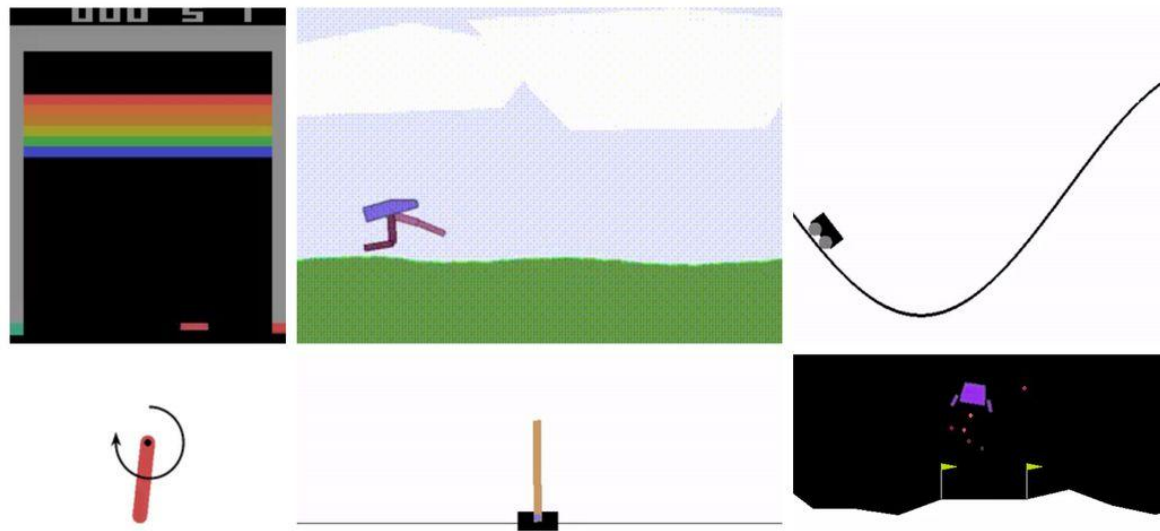
```
pip install gymnasium  
pip install pyglet  
pip install pygame
```

```
import gymnasium as gym
```

Gymnasium for Reinforcement Learning

8

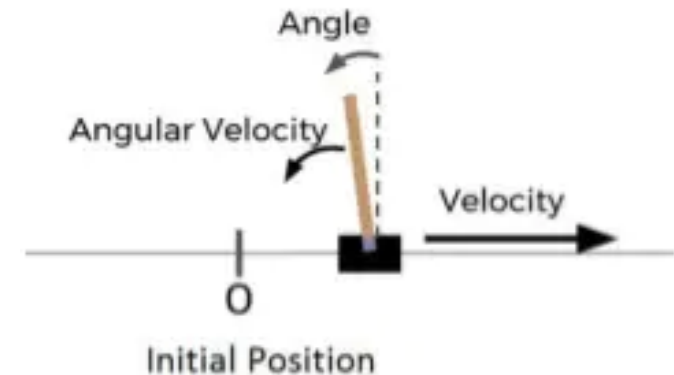
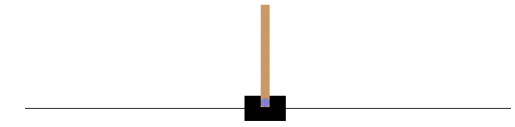
- OpenAI's Gymnasium is an API standard for reinforcement learning, offering a diverse collection of reference environments.
- The Gymnasium interface is simple and intuitive, and capable of representing general RL problems.
- It has grown in popularity tremendously within the RL community.



Gymnasium Example: Cart Pole

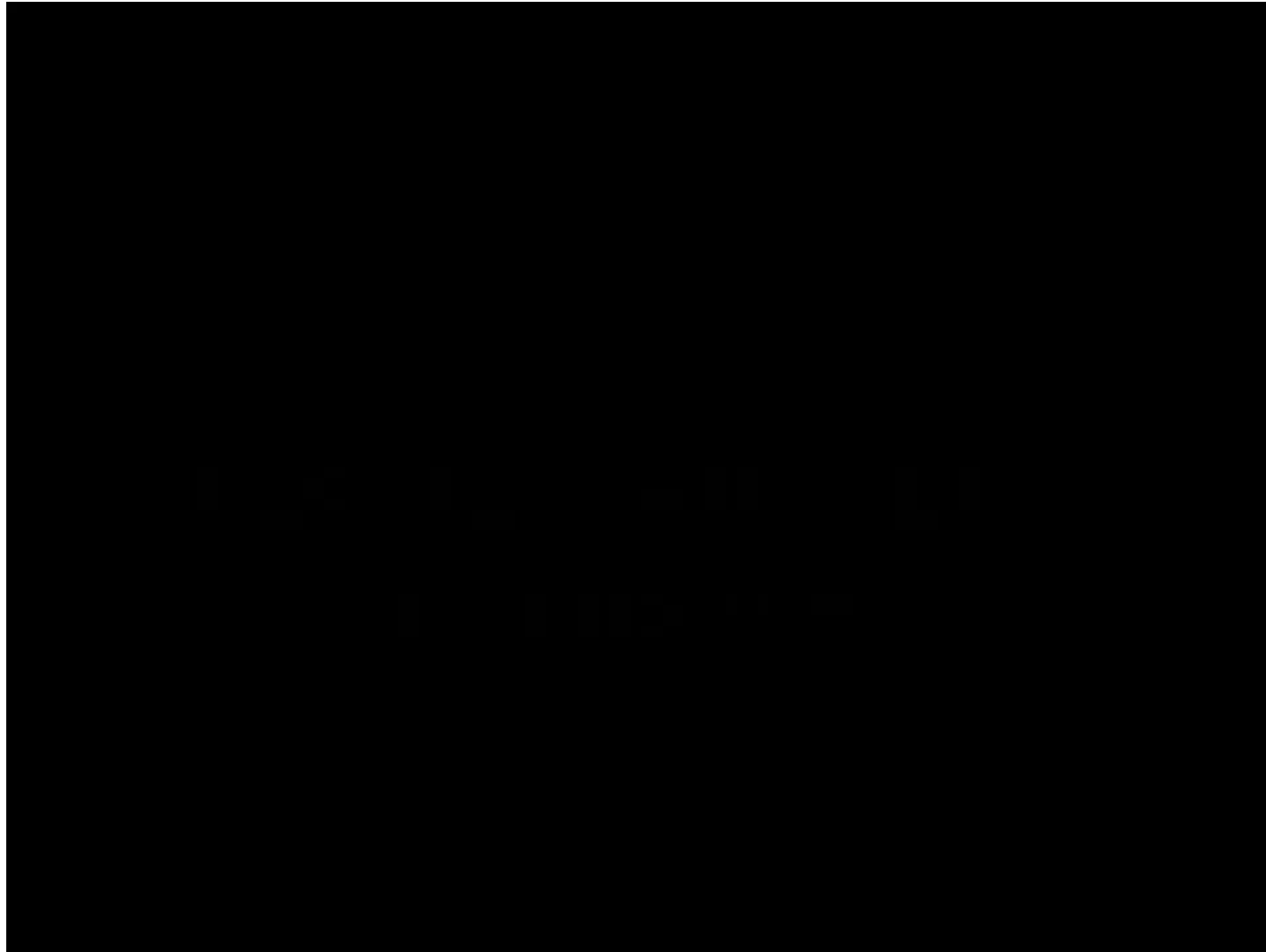
9

- In the [CartPole-v1](#) example, a **pole** is attached to a cart via an un-actuated joint and moves along a frictionless track.
- The **pendulum** is placed upright on the cart, and the objective is to keep the pole upright by applying forces to the left and right of the cart.
- A **reward** of +1 is provided for each time step during which the pole remains upright, with a default reward threshold of 500.
- The **episode** will **end** if the pole angle exceeds $\pm 12^\circ$ from vertical, the cart moves more than 2.4 units from the centre or the episode length exceeds 500.
- The **action space** is a `numpy` array whose values indicate the direction of the fixed force with which the cart is pushed:
 - `0`: push the cart to the left
 - `1`: push the cart to the right
- The **observation space** is a `numpy` array containing 4 floating-point values:
 - Cart Position
 - Cart Velocity
 - Pole Angle
 - Pole Angular Velocity



Gymnasium Example: Cart Pole - Simulation

10



Gymnasium's Functions

11

- `make()`: creates the environment.
- `reset()`: sets the environment to the default starting state.
- `render()`: creates a pop-up window to display the bot interacting with the environment.
- `step()`: the action taken by the bot. It returns an observation in the `numpy` array format `<observations, reward, done, info>`
- `sample()`: provides random input samples for the bot.
- `close()`: closes the environment after an action has been performed.

```
env = gym.make("CartPole-v1", render_mode = "human")
env.reset()
for _ in range(200):
    env.render()
    env.reset()
    env.step(env.action_space.sample())
env.close()
```

environment
initialisation

200 time-steps are
executed and
rendered

bot performs a random
action based on the
possible actions available
in the environment

Gymnasium's Observations

12

Observations are environment-specific information variables:

- `observation(object)`: an object representing an observation of the environment. E.g., joint angles and joint velocities of a robot or the board state in a board game.
- `reward(float)`: the amount of reward achieved by the previous action. The scale varies between environments, but the goal is always to maximise the total reward.
- `terminated(boolean)`: indicates whether a terminal state has been reached. Most tasks are divided into well-defined episodes, and `terminated` being true indicates that the episode has ended. E.g., the pole may have tipped too far or the bot may have lost its last life.
- `truncated(bool)`: whether a truncation condition has been satisfied. In this case, when the episode length is greater than 500. This can be used to end the episode prematurely, before a terminal state is reached.
- `info(dict)`: diagnostic information that is useful for debugging. E.g., it can contain the raw probabilities behind the environment's last state change.

Gymnasium's Observations

13

The process begins by calling `reset()`, which returns an initial observation.

The first code can be written more properly with respect to the episodes and `done` flag:

```
env = gym.make("CartPole-v1", render_mode = "human")
env.reset()
for _ in range(200):
    env.render()
    env.reset()
    env.step(env.action_space.sample())
env.close()
```

```
env = gym.make("CartPole-v1", render_mode = "human")
env.reset()
```

```
for i_episode in range(20):
```

definition of the number of episodes

```
    observation = env.reset()
```

```
    for t in range(30):
```

definition of the number of time steps per episode

```
        env.render()
```

```
        print(observation)
```

```
        action = env.action_space.sample()
```

```
        observation, reward, terminated, truncated, info = env.step(action)
```

```
        if terminated:
```

```
            print("Episode finished after {} time steps".format(t+1))
```

```
            break
```

```
env.close()
```

bot perception for each step based
on the action taken

verify if the episode has finished

Hard-Coded Policy

14

In order to keep the pole upright, a **policy** or strategy needs to be devised that the agent can follow in order to maintain balance at each stage. The agent can use all past actions and observations to inform their next decision.

By observing the game, it can be seen that if the pole leans to the right, the cart should be moved to the right. If the pole tilts to the left, the cart should be moved to the left.

definition of hard-coded policies

move the cart to the right if the pole is falling to the right

angle is measured from a straight vertical line

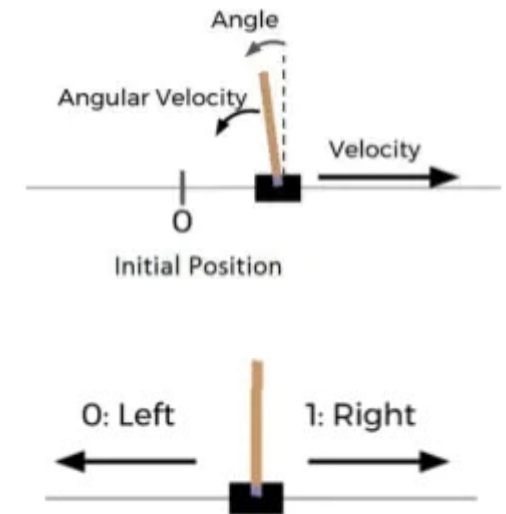
perform action

```
cart_pos, cart_vel, pole_ang, ang_vel = observation

env = gym.make("CartPole-v1", render_mode = "human")
env.reset()

for t in range(20):
    env.render()
    print(observation)
    cart_pos, cart_vel, pole_ang, ang_vel = observation
    if pole_ang > 0:
        action = 1  # move right
    else:
        action = 0  # move left
    observation, reward, terminated, truncated, info = env.step(action)

env.close()
```



Environment

15

Global hyperparameters

```
EPISODES = 5000  
DISCOUNT = 0.95  
EPISODE_DISPLAY = 500  
LEARNING_RATE = 0.25  
EPSILON = 0.2
```

Number of episodes: used to train the RL model.

Discount factor: used to measure how far ahead in time the algorithm must look. E.g., if `factor = 0`, none of the future rewards are considered in Q-learning. If `factor = 1`, future rewards are given a high weighting.

Episode display: defines the number of episodes necessary to run before rendering an episode. E.g., episodes 0, 500, 1000, 1500, .. are rendered. This is useful for visually verifying the RL's model evolution.

Learning rate: set between 0 and 1 to facilitate the Q-value updates at the desired rate. E.g., if `rate = 0`, the Q-values are never updated and nothing is learnt. If `rate = 1`, nothing is added to the current Q-value.

Exploration constant: used to give the bot an element of exploration. E.g., if `epsilon = 0`, the algorithm only considers actions corresponding to the highest Q-value. If `epsilon = 1`, the algorithm only selects random action values.

Environment

16

Prepare the environment and examine the observation and action spaces.

```
def prepare_env():  
    env = gym.make("CartPole-v1")  
  
    print('Env. Observation Space: ', env.observation_space)  
    print('Env. Observation Space - High:', env.observation_space.high)  
    print('Env. Observation Space - Low:', env.observation_space.low)  
  
    print('Env. Action Space:', env.action_space)  
    print('Env. Actions Space:', env.action_space.n)  
    return env
```

Environment values

Observation Space:

- [0] cart position along x-axis
- [1] cart velocity
- [2] pole angle (rad)
- [3] pole angular velocity

continuous **min** and **max** values for each observation variable
[position of cart, velocity of cart, angle of pole,
rotation rate of pole]

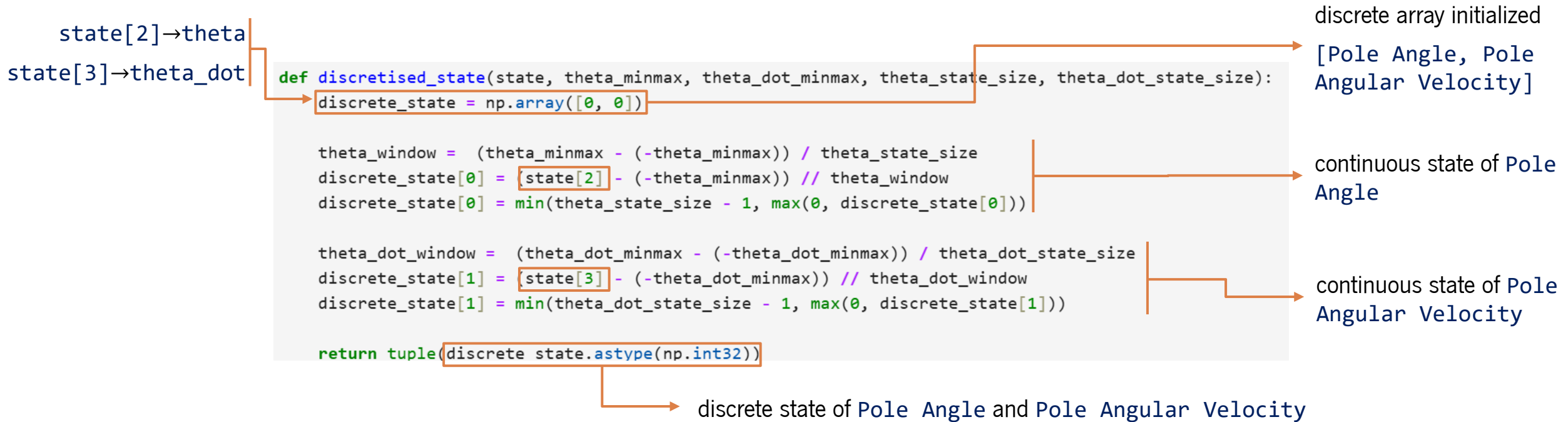
Action Space:

- [0] push cart to the left
- [1] push cart to the right

Discretize State's Results

17

When the `step()` function is executed, it returns a continuous state, which is then converted into a discrete state by the `discretised_state(state)` function. The **Pole Angle** and **Pole Angular Velocity** features will be used to train the RL model. In this case, the number of possibilities will be split into 50 bins.



Q-Learning

18

Q-table of size: `theta_state_size x theta_dot_state_size x env.action_space.n`

```
def train_cart_pole_qlearning(EPIISODES, DISCOUNT, EPISODE_DISPLAY, LEARNING_RATE, EPSILON):  
    env = prepare_env()  
  
    theta_minmax = env.observation_space.high[2]  
    theta_dot_minmax = math.radians(50)  
    theta_state_size = 50  
    theta_dot_state_size = 50  
  
    Q_TABLE = np.random.randn(theta_state_size, theta_dot_state_size, env.action_space.n)  
  
    ep_rewards = []  
    ep_rewards_table = {'ep': [], 'avg': [], 'min': [], 'max': []}
```

Use the **min** and **max** observations to convert the continuous states of the features **Pole Angle** and **Pole Angular Velocity** into discrete states

50 Pole Angle states

50 Pole Angular Velocity states

Q-table is initialised with random values and is used to calculate the maximum expected future reward for each action in each state. The Q-table dimension varies depending on:

- **Environment possible actions** (2) - left & right
- **Environment number of states** (50 pole angle states, 50 pole angular velocity states) – *increased number of states provides a higher resolution of the state space*

dict model stats to verify model learning progression

Q-Learning

19

```
for episode in range(EPIISODES):
    episode_reward = 0
    terminated = False
    i = 0
    if episode % EPISODE_DISPLAY == 0:
        render_state = True
    else:
        render_state = False
    curr_discrete_state = discretised_state(env.reset()[0],
                                           theta_minmax, theta_dot_minmax,
                                           theta_state_size, theta_dot_state_size)
```

Variables are initialised at the start of an episode

Q-Learning

20

```
while not terminated:
    if np.random.random() > EPSILON:
        action = np.argmax(Q_TABLE[curr_discrete_state])
    else:
        action = np.random.randint(0, env.action_space.n)

    new_state, reward, terminated, _, _ = env.step(action)
    new_discrete_state = discretised_state(new_state,
                                           theta_minmax, theta_dot_minmax,
                                           theta_state_size, theta_dot_state_size)

    if render_state:
        env.render()

    if not terminated:
        max_future_q = np.max(Q_TABLE[new_discrete_state[0], new_discrete_state[1]])
        current_q = Q_TABLE[curr_discrete_state[0], curr_discrete_state[1], action]
        new_q = current_q + LEARNING_RATE * (reward + DISCOUNT * max_future_q - current_q)
        Q_TABLE[curr_discrete_state[0], curr_discrete_state[1], action] = new_q
    i += 1
    curr_discrete_state = new_discrete_state
    episode_reward += reward
```

Based on the **exploration constant**, select a random action or the action with the highest Q-value

Bot then executes the selected action and acquires an observation from the new state

If the episode is not yet completed, update the Q-table using the Q-learning formula:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

Update the `current_state` and `episode_reward` until the end of the episode

Q-Learning

21

```
ep_rewards.append(episode_reward)

if not episode % EPISODE_DISPLAY:
    avg_reward = sum(ep_rewards[-EPISODE_DISPLAY:]) / len(ep_rewards[-EPISODE_DISPLAY:])
    ep_rewards_table['ep'].append(episode)
    ep_rewards_table['avg'].append(avg_reward)
    ep_rewards_table['min'].append(min(ep_rewards[-EPISODE_DISPLAY:]))
    ep_rewards_table['max'].append(max(ep_rewards[-EPISODE_DISPLAY:]))
    print(f"Episode:{episode} avg:{avg_reward} min:{min(ep_rewards[-EPISODE_DISPLAY:])}
          max:{max(ep_rewards[-EPISODE_DISPLAY:])}")
```

Save the `episode_reward` for the model learning analysis

Add the episode information to the episode rewards table

Q-Learning

22

```
env.close()

plt.plot(ep_rewards_table['ep'], ep_rewards_table['avg'], label = "avg")
plt.plot(ep_rewards_table['ep'], ep_rewards_table['min'], label = "min")
plt.plot(ep_rewards_table['ep'], ep_rewards_table['max'], label = "max")
plt.legend(loc = 4)
plt.title('CartPole Q-Learning')
plt.ylabel('Average reward/Episode')
plt.xlabel('Episodes')
plt.show()

return ep_rewards_table
```

Plot model evolution performance: generate a plot based on the episode rewards table to verify the evolution of the episode rewards for each episode

Q-Learning

23

```
ep_rewards_table_qlearning = train_cart_pole_qlearning(EPIISODES, DISCOUNT, EPISODE_DISPLAY, LEARNING_RATE, EPSILON)
```

```
Env. Observation Space: Box([-4.8          -inf -0.41887903      -inf],
```

```
[4.8          inf 0.41887903      inf], (4,)), float32)
```

```
Env. Observation Space - High: [4.8          inf 0.41887903      inf]
```

```
Env. Observation Space - Low: [-4.8          -inf -0.41887903      -inf]
```

```
Env. Action Space: Discrete(2)
```

```
Env. Actions Space: 2
```

```
Episode:0 avg:21.0 min:21.0 max:21.0
```

```
Episode:500 avg:18.924 min:8.0 max:97.0
```

```
Episode:1000 avg:17.36 min:8.0 max:83.0
```

```
Episode:1500 avg:18.566 min:8.0 max:100.0
```

```
Episode:2000 avg:25.388 min:8.0 max:104.0
```

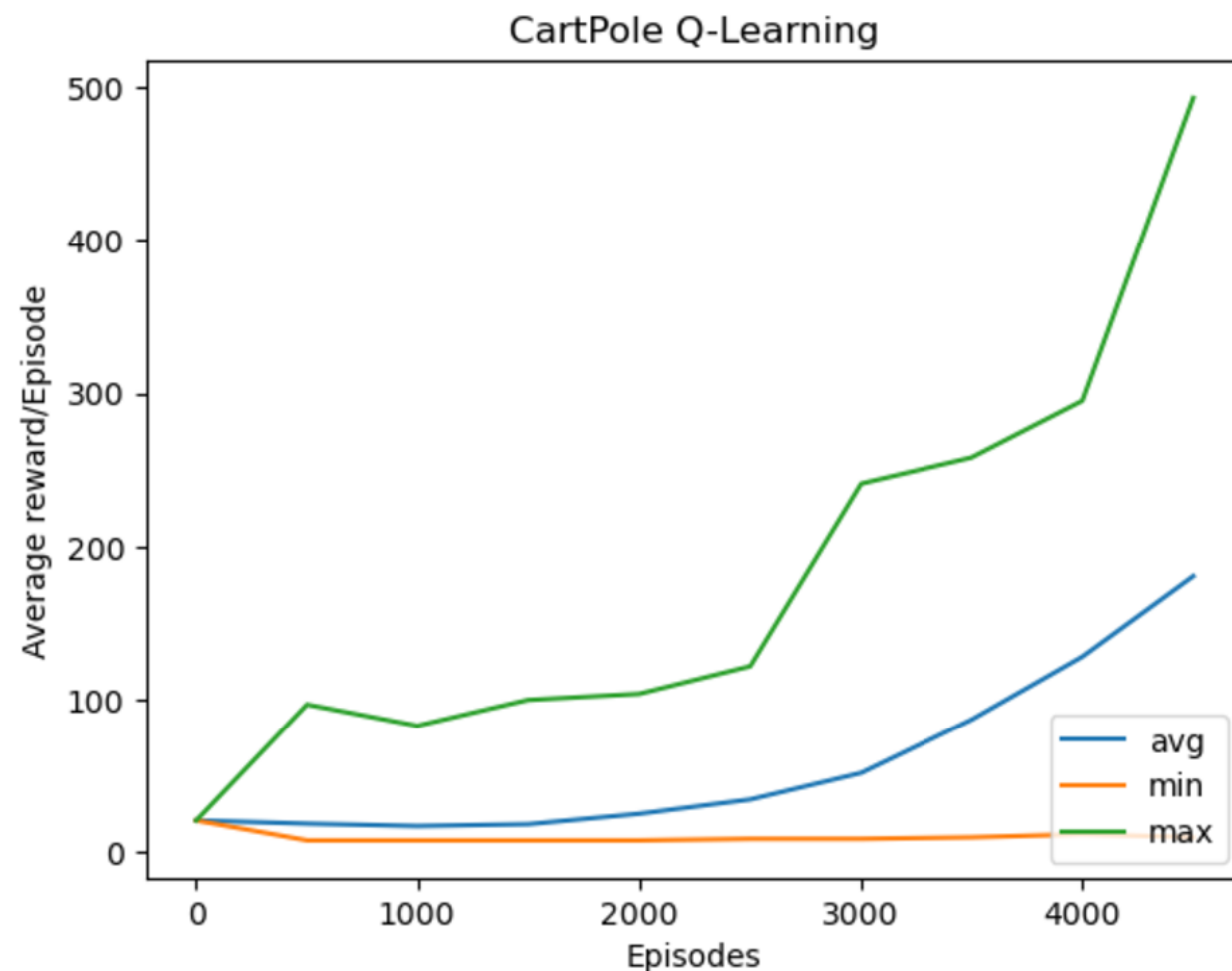
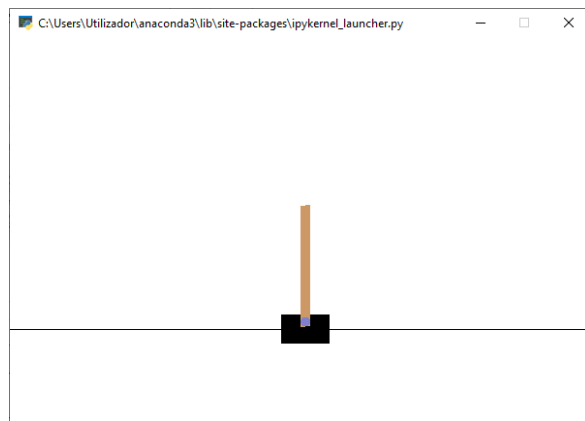
```
Episode:2500 avg:34.716 min:9.0 max:122.0
```

```
Episode:3000 avg:52.078 min:9.0 max:241.0
```

```
Episode:3500 avg:86.98 min:10.0 max:258.0
```

```
Episode:4000 avg:128.21 min:12.0 max:295.0
```

```
Episode:4500 avg:180.806 min:10.0 max:493.0
```



SARSA

24

```
def train_cart_pole_sarsa(EPIISODES, DISCOUNT, EPISODE_DISPLAY, LEARNING_RATE, EPSILON):
    env = prepare_env()

    theta_minmax = env.observation_space.high[2]
    theta_dot_minmax = math.radians(50)
    theta_state_size = 50
    theta_dot_state_size = 50

    Q_TABLE = np.random.randn(theta_state_size, theta_dot_state_size, env.action_space.n)

    ep_rewards = []
    ep_rewards_table = {'ep': [], 'avg': [], 'min': [], 'max': []}
    for episode in range(EPIISODES):
        episode_reward = 0
        terminated = False
        if episode % EPISODE_DISPLAY == 0:
            render_state = True
        else:
            render_state = False
        curr_discrete_state = discretised_state(env.reset()[0],
                                                theta_minmax, theta_dot_minmax,
                                                theta_state_size, theta_dot_state_size)

        if np.random.random() > EPSILON:
            action = np.argmax(Q_TABLE[curr_discrete_state])
        else:
            action = np.random.randint(0, env.action_space.n)
```

The preparation of the environment for applying SARSA remains the same.

SARSA

25

```
while not terminated:
    new_state, reward, terminated, _, _ = env.step(action)
    new_discrete_state = discretised_state(new_state,
                                           theta_minmax, theta_dot_minmax,
                                           theta_state_size, theta_dot_state_size)

    if np.random.random() > EPSILON:
        new_action = np.argmax(Q_TABLE[new_discrete_state])
    else:
        new_action = np.random.randint(0, env.action_space.n)

    if render_state:
        env.render()

    if not terminated:
        current_q = Q_TABLE[curr_discrete_state + (action,)]
        max_future_q = Q_TABLE[new_discrete_state + (new_action,)]
        new_q = current_q + LEARNING_RATE * (reward + DISCOUNT * max_future_q - current_q)
        Q_TABLE[curr_discrete_state + (action,)] = new_q
    curr_discrete_state = new_discrete_state
    action = new_action
    episode_reward += reward
```

Based on the **exploration constant**, select a random action or the action with the highest Q value for the **next state**

If the episode is not yet complete, update the Q table using the SARSA formula.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

SARSA

26

```
ep_rewards.append(episode_reward)

if not episode % EPISODE_DISPLAY:
    avg_reward = sum(ep_rewards[-EPISODE_DISPLAY:]) / len(ep_rewards[-EPISODE_DISPLAY:])
    ep_rewards_table['ep'].append(episode)
    ep_rewards_table['avg'].append(avg_reward)
    ep_rewards_table['min'].append(min(ep_rewards[-EPISODE_DISPLAY:]))
    ep_rewards_table['max'].append(max(ep_rewards[-EPISODE_DISPLAY:]))
    print(f"Episode:{episode} avg:{avg_reward} min:{min(ep_rewards[-EPISODE_DISPLAY:])} max:{max(ep_rewards[-EPISODE_DISPLAY:])}")

env.close()

plt.plot(ep_rewards_table['ep'], ep_rewards_table['avg'], label = "avg")
plt.plot(ep_rewards_table['ep'], ep_rewards_table['min'], label = "min")
plt.plot(ep_rewards_table['ep'], ep_rewards_table['max'], label = "max")
plt.legend(loc = 4)
plt.title('CartPole SARSA')
plt.ylabel('Average reward/Episode')
plt.xlabel('Episodes')
plt.show()

return ep_rewards_table
```

SARSA

27

```
ep_rewards_table_sarsa = train_cart_pole_sarsa(EPIISODES, DISCOUNT, EPISODE_DISPLAY, LEARNING_RATE, EPSILON)
```

```
Env. Observation Space: Box([-4.8          -inf -0.41887903      -inf],  
[4.8          inf 0.41887903      inf], (4,), float32)
```

```
Env. Observation Space - High: [4.8          inf 0.41887903      inf]
```

```
Env. Observation Space - Low: [-4.8          -inf -0.41887903      -inf]
```

```
Env. Action Space: Discrete(2)
```

```
Env. Actions Space: 2
```

```
Episode:0 avg:19.0 min:19.0 max:19.0
```

```
Episode:500 avg:17.448 min:8.0 max:67.0
```

```
Episode:1000 avg:17.814 min:8.0 max:89.0
```

```
Episode:1500 avg:22.402 min:8.0 max:113.0
```

```
Episode:2000 avg:23.406 min:8.0 max:88.0
```

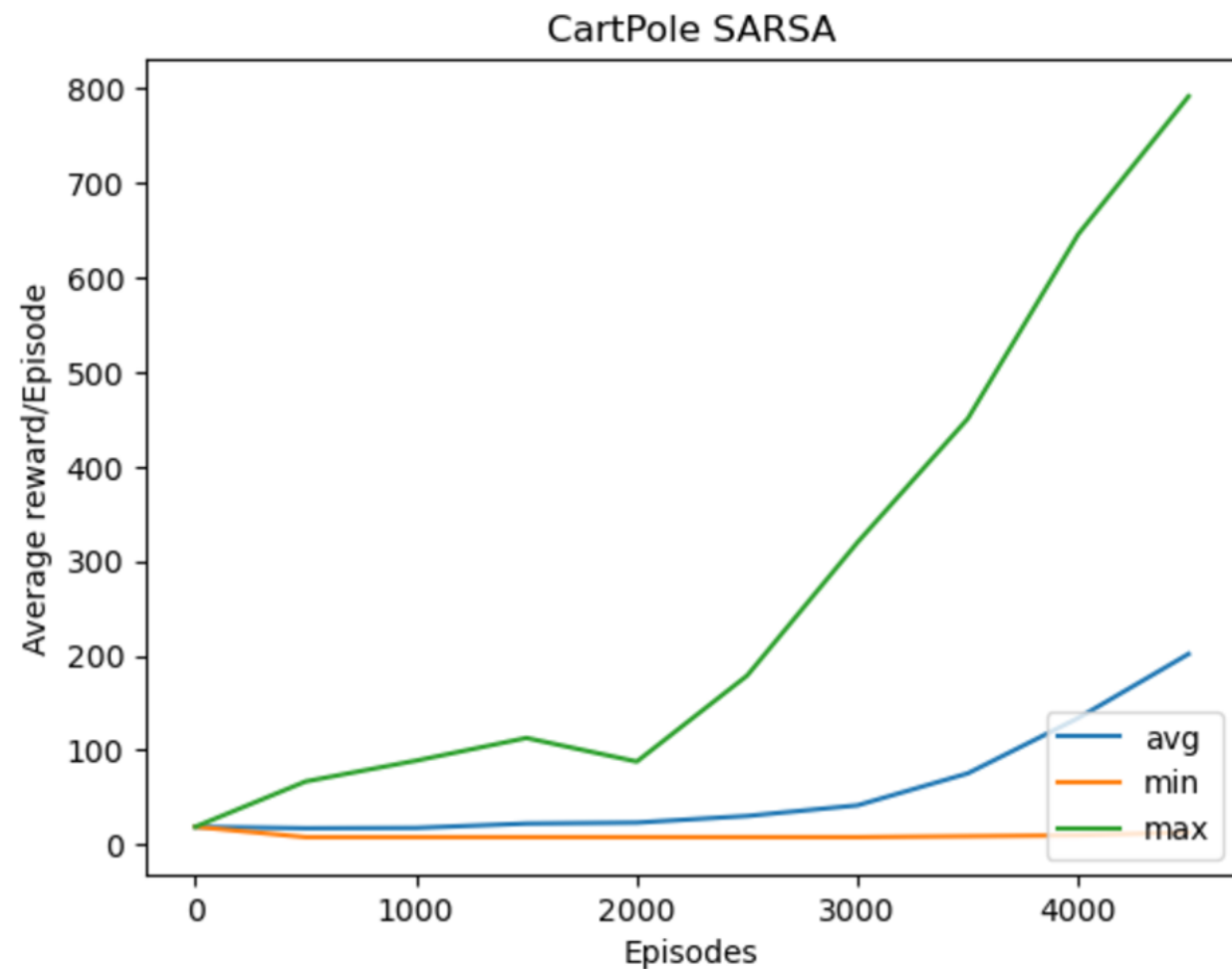
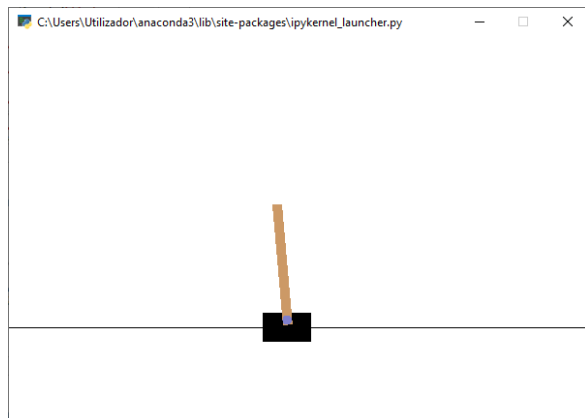
```
Episode:2500 avg:30.432 min:8.0 max:179.0
```

```
Episode:3000 avg:41.69 min:8.0 max:320.0
```

```
Episode:3500 avg:75.542 min:9.0 max:451.0
```

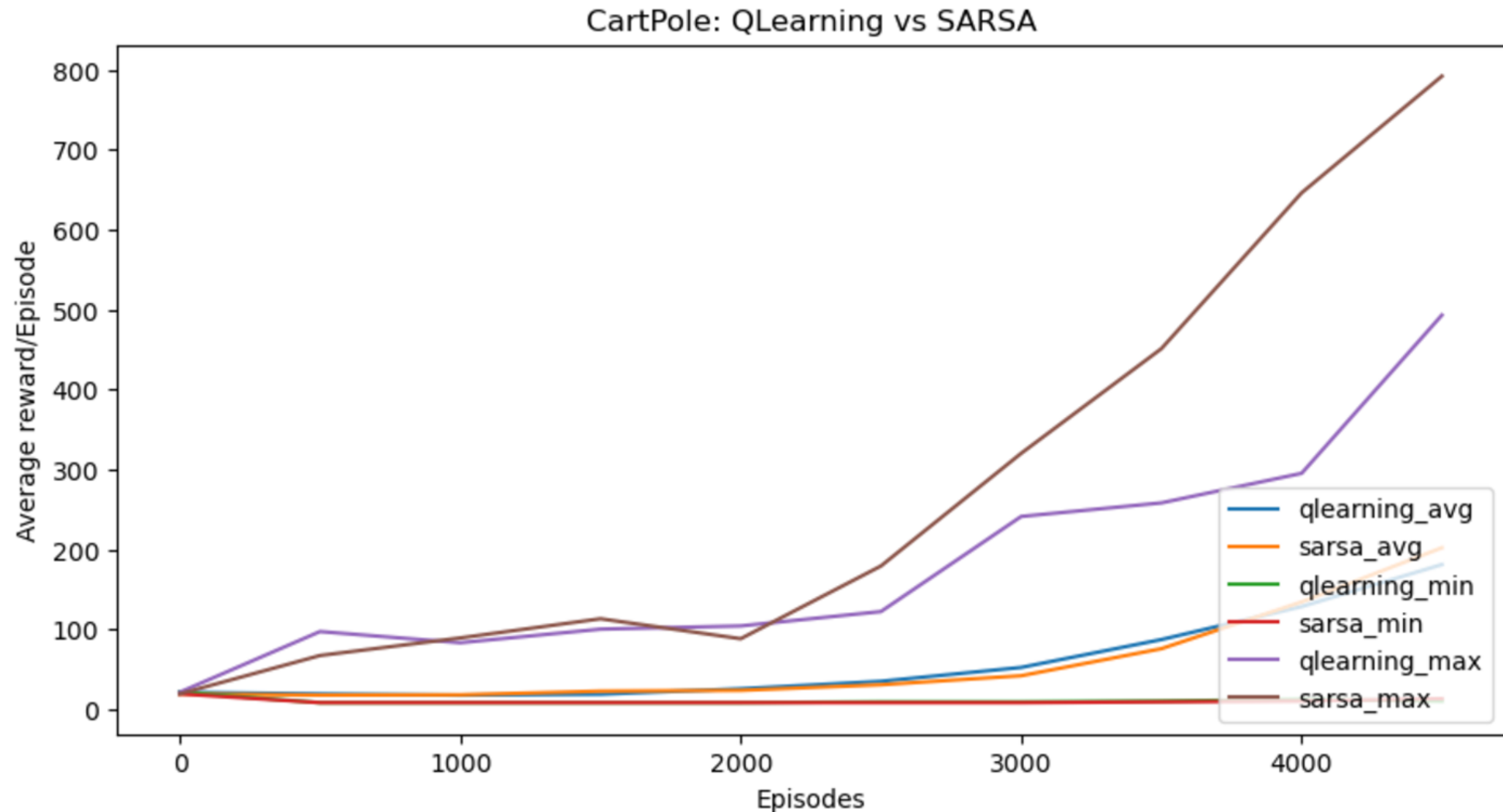
```
Episode:4000 avg:134.048 min:10.0 max:646.0
```

```
Episode:4500 avg:201.876 min:13.0 max:792.0
```



Q-Learning vs. SARSA

28



Q-Learning vs. SARSA

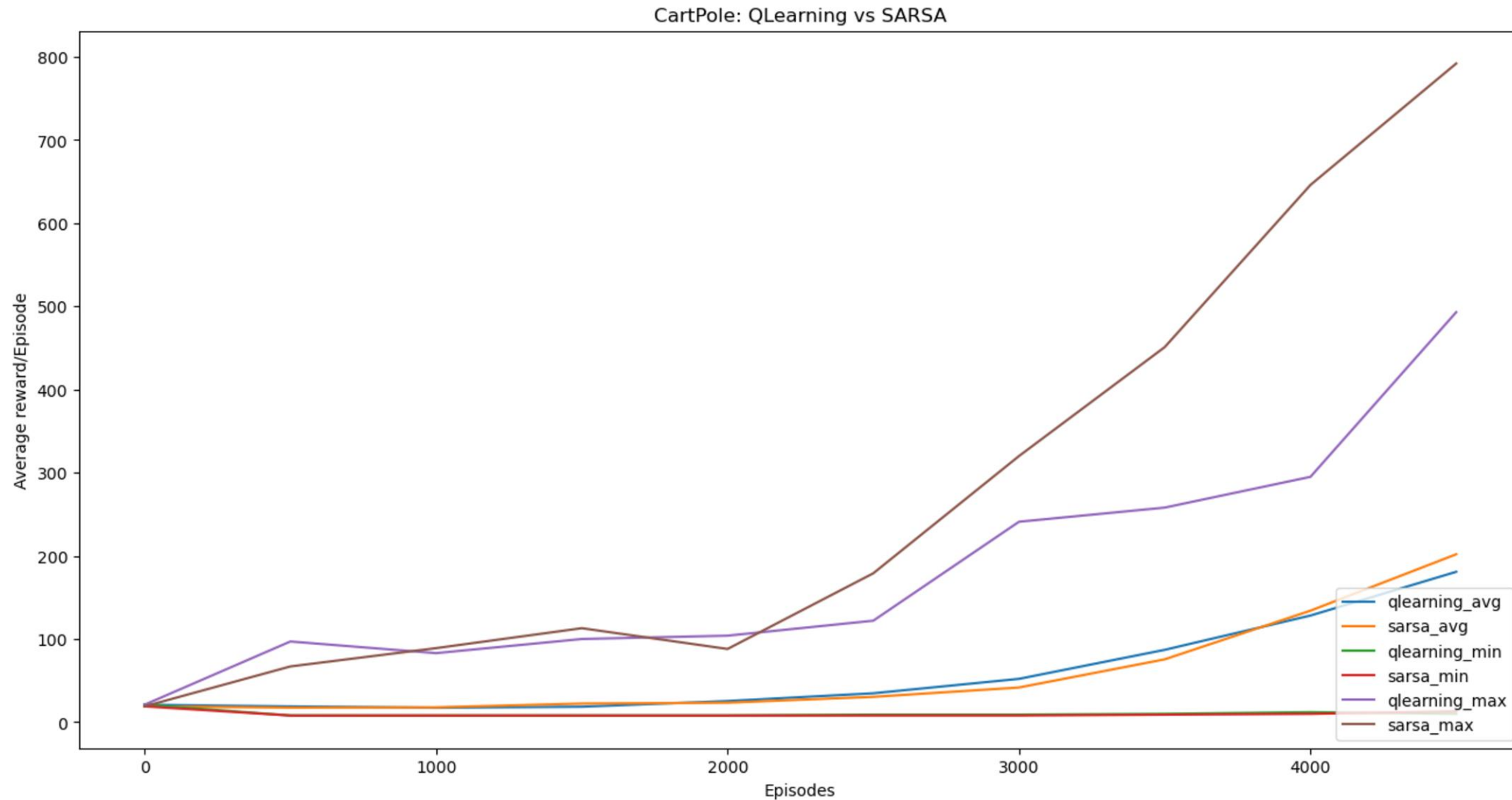
29

When the graphs of **SARSA** and **Q-Learning** are compared, it can be seen that:

- For this run, the reward converges to a higher value with SARSA than with Q-Learning. This may be due to the action selection step, in which an epsilon-greedy policy is applied. In Q-Learning, the action corresponding to the largest Q-value is selected.
- The agent obtains the maximum reward in 4500 episodes for Q-Learning and in the case of the cart-pole for SARSA.
- Training both models with more episodes and optimising their hyperparameters could further improve decision-making performance. Further experiments could be conducted by adapting the input features and changing the number of states per feature.

Q-Learning vs. SARSA

30





Hands On